ACADEMIC SUBMISSION LAB PORTFOLIO

PYTHON PROGRAMMING CASE STUDY COMPILATION

Activity-Based Learning: 10 Selected Beginner Projects with Logic Flowcharts

A comprehensive collection of 10 easy-level Python case studies featuring problem specifications, complete source code, visual execution flowcharts, and verified output reports.

INDEX OF SELECTED CASE STUDIES

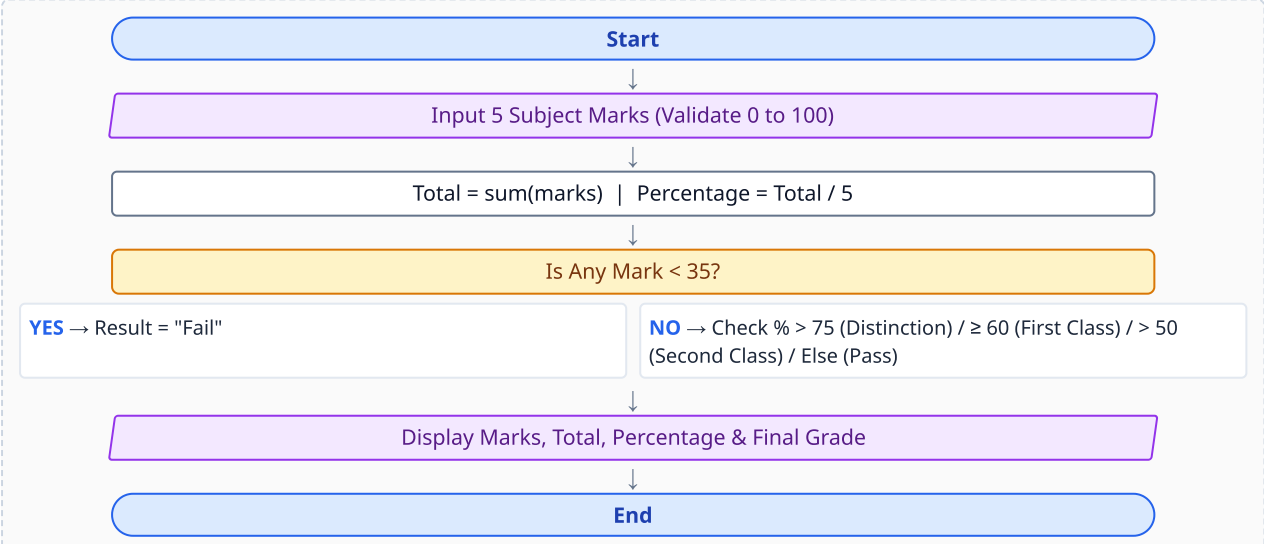
Index	Case Study Title	Syllabus Scope & Logic Included
CS-01	Student Marks & Result Analysis	Conditionals, Comparison Operators, List Aggregation
CS-02	Progressive Electricity Billing	Slab Calculation, Branching ('if-elif'), Input Validation
CS-03	ATM Transaction Simulation	State Loops ('while'), Menu Structures, Error Handling
CS-04	Student Attendance Eligibility	Threshold Calculations, Predictive Iterative Loops
CS-05	Simple Retail Billing System	Tuples, List Storage, Loop Accumulation, Discount Flags
CS-06	Employee Salary Slip Calculator	Fixed Formulas, Percentage Additions, Deductions
CS-07	Password Policy Validation Engine	String Traversal, Built-in Inspection Methods, Dictionaries
CS-08	Library Late Fine Reconciliation	Multi-tier Slab Calculations, Overdue Penalties
CS-09	Fuel Consumption & Route Costing	User-Defined Functions, Input Validation Helpers
CS-10	Bus Passenger Occupancy Simulation	List Iteration, Real-Time Accumulation, Capacity Flags

CASE STUDY 1: STUDENT MARKS ANALYSIS

Conditionals & Lists

Problem Statement: Evaluate academic performance across 5 subjects with input validation, aggregate totals, percentage calculation, and class grade assignment.

LOGIC & DECISION FLOWCHART



PYTHON SOURCE CODE

```
subjects = ["English", "Mathematics", "Physics", "Programming", "Chemistry"] marks = [] for subject in subjects: while True: try: mark = float(input(f"Enter {subject} marks (0-100): ")) if 0 <= mark <= 100: marks.append(mark) break print("Marks must be between 0 and 100.") except ValueError: print("Please enter a valid number.") total = sum(marks) percentage = total / len(subjects) if any(mark < 35 for mark in marks): result = "Fail" elif percentage > 75: result = "Distinction" elif percentage >= 60: result = "First Class" elif percentage > 50: result = "Second Class" else: result = "Pass" print(f"\nTotal: {total:.1f}/500 | Percentage: {percentage:.2f}% | Result: {result}")
```

EXECUTION OUTPUT

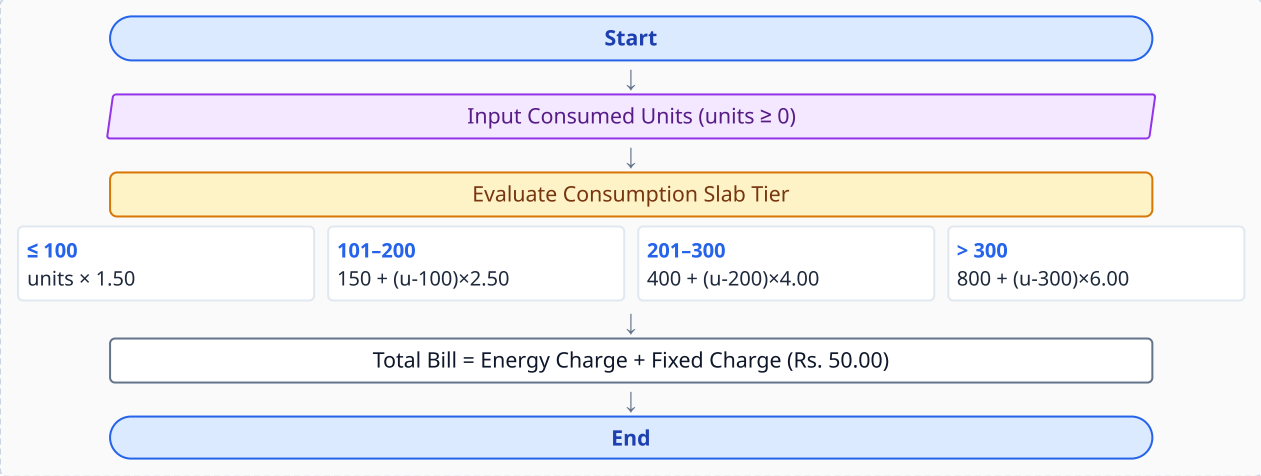
```
Enter English marks (0-100): 85 | Enter Mathematics marks: 92 | Enter Physics marks: 78 Enter Programming marks: 95 | Enter Chemistry marks: 88 Total: 438.0/500 | Percentage: 87.60% | Result: Distinction
```

CASE STUDY 2: ELECTRICITY BILL CALCULATOR

Multi-Slab Calculations

Problem Statement: Compute electricity charges using progressive tier slabs (1.50, 2.50, 4.00, 6.00 per unit) plus a fixed standing fee.

LOGIC & SLAB FLOWCHART



PYTHON SOURCE CODE

```
units = float(input("Enter electricity units consumed: ")) if units <= 100: energy_charge = units * 1.50 elif units <= 200: energy_charge = (100 * 1.50) + ((units - 100) * 2.50) elif units <= 300: energy_charge = (100 * 1.50) + (100 * 2.50) + ((units - 200) * 4.00) else: energy_charge = (100 * 1.50) + (100 * 2.50) + (100 * 4.00) + ((units - 300) * 6.00) fixed_charge = 50.0 total_bill = energy_charge + fixed_charge print(f"Units: {units:.1f} | Energy Charge: Rs.{energy_charge:.2f} | Total: Rs.{total_bill:.2f}")
```

EXECUTION OUTPUT

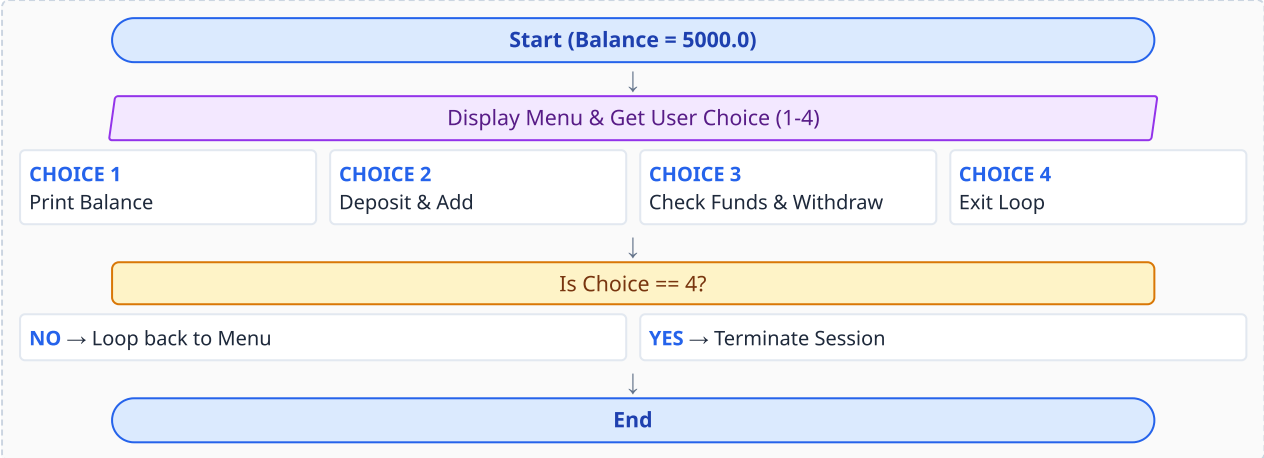
```
Enter electricity units consumed: 250 Units: 250.0 | Energy Charge: Rs.600.00 | Fixed Charge: Rs.50.00 | Total: Rs.650.00
```

CASE STUDY 3: ATM TRANSACTION SIMULATION

Interactive Loop

Problem Statement: Simulate interactive banking transactions including balance inquiry, deposits, and withdrawal validations.

LOGIC & LOOP FLOWCHART



PYTHON SOURCE CODE

```
balance = 5000.0 while True: print("\n1. Check balance | 2. Deposit | 3. Withdraw | 4. Exit") choice = input("Enter choice (1-4): ") if choice == "1": print(f"Balance: Rs. {balance:.2f}") elif choice == "2": amount = float(input("Enter deposit: ")) if amount > 0: balance += amount elif choice == "3": amount = float(input("Enter withdrawal: ")) if amount <= balance: balance -= amount else: print("Insufficient funds.") elif choice == "4": print("Thank you for using ATM.") break
```

EXECUTION OUTPUT

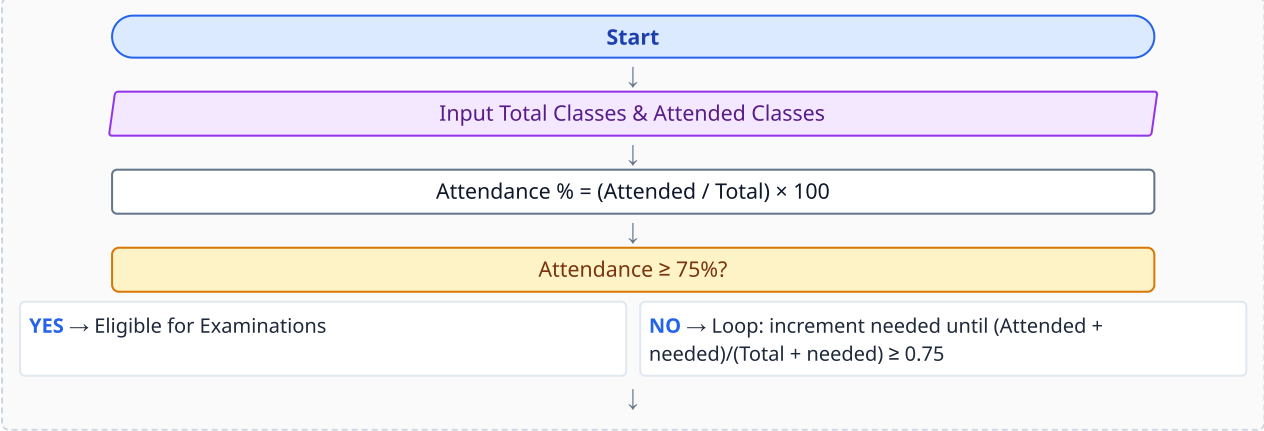
```
===== ATM MENU ===== Choice: 3 | Enter withdrawal amount: Rs. 1200 Please collect cash. Remaining balance: Rs. 3800.00
```

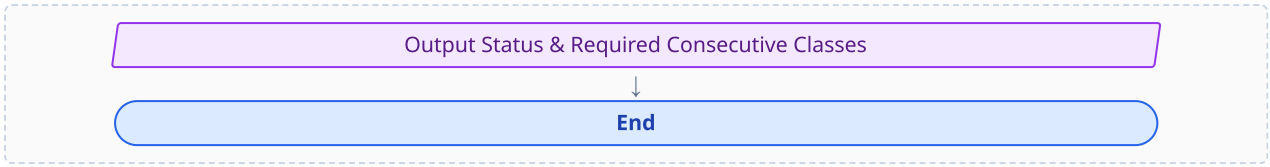
CASE STUDY 4: STUDENT ATTENDANCE CALCULATOR

Threshold Computation

Problem Statement: Compute attendance percentage and dynamically calculate future consecutive classes required to reach 75% eligibility.

LOGIC & CALCULATION FLOWCHART





PYTHON SOURCE CODE

```
total_classes = int(input("Enter total classes held: ")) attended_classes = int(input("Enter classes attended: ")) attendance = (attended_classes / total_classes) * 100 print(f"Attendance: {attendance:.2f}%") if attendance >= 75: print("Status: Eligible") else: needed = 0 while ((attended_classes + needed) / (total_classes + needed)) < 0.75: needed += 1 print(f"Required: Attend next {needed} consecutive classes for 75%.")
```

EXECUTION OUTPUT

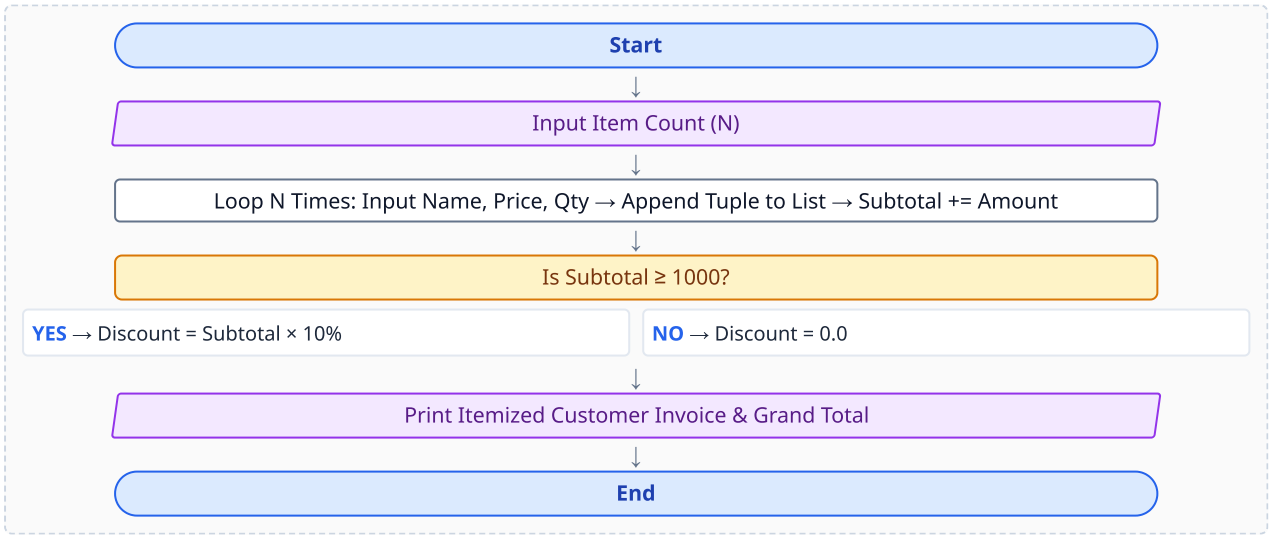
```
Total: 40 | Attended: 26 | Attendance: 65.00% Status: NOT ELIGIBLE (Shortage) | Required: Attend next 16 consecutive classes.
```

CASE STUDY 5: RETAIL BILLING SYSTEM

Tuples & Discounts

Problem Statement: Record items into a list of tuples, calculate subtotal, apply a 10% discount on orders exceeding Rs. 1000, and print receipt.

LOGIC & ACCUMULATION FLOWCHART



PYTHON SOURCE CODE

```
items = [] subtotal = 0.0 count = int(input("How many items? ")) for i in range(count): name = input("Item Name: ") price = float(input("Price: ")) qty = int(input("Qty: ")) amount = price * qty items.append((name, price, qty, amount)) subtotal += amount discount = subtotal * 0.10 if subtotal >= 1000 else 0.0 final_total = subtotal - discount print(f"Subtotal: Rs. {subtotal:.2f} | Discount: Rs.{discount:.2f} | Total: Rs.{final_total:.2f}")
```

EXECUTION OUTPUT

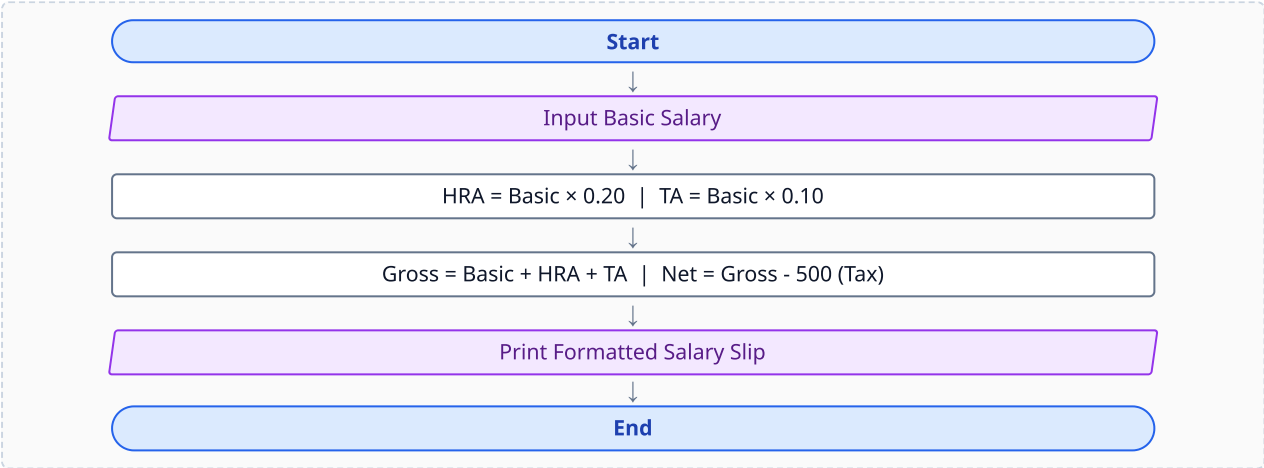
```
Notebook (5 x Rs.60.00) = Rs.300.00 | Pen Drive (1 x Rs.850.00) = Rs.850.00 Subtotal: Rs. 1150.00 | Discount: Rs. 115.00 | Grand Total: Rs. 1035.00
```

CASE STUDY 6: EMPLOYEE SALARY CALCULATOR

Formulas & Formatting

Problem Statement: Compute employee gross earnings from Basic, HRA (20%), and Travel Allowance (10%), apply deductions, and output net salary.

LOGIC FLOWCHART



PYTHON SOURCE CODE

```
basic_salary = float(input("Enter basic salary (Rs.): ")) hra = basic_salary * 0.20 travel_allowance = basic_salary * 0.10 gross_salary = basic_salary + hra + travel_allowance deduction = 500.0 net_salary = gross_salary - deduction print(f"Gross: Rs.{gross_salary:.2f} | Tax: Rs.{deduction:.2f} | Net Pay: Rs.{net_salary:.2f}")
```

EXECUTION OUTPUT

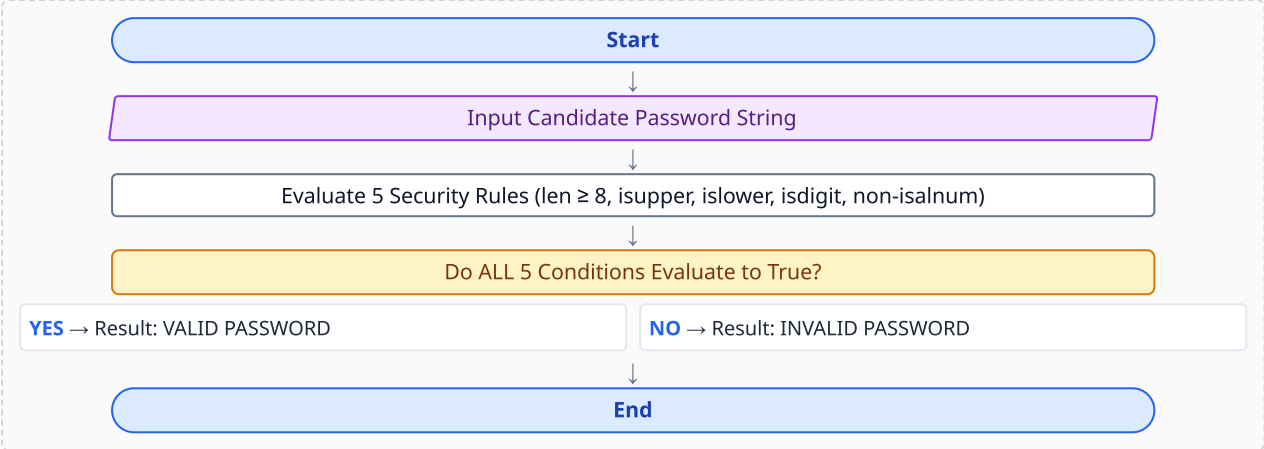
```
Enter basic salary: 35000 Basic: Rs.35000.00 | HRA: Rs.7000.00 | TA: Rs.3500.00 | Gross: Rs.45500.00 | Net: Rs.45000.00
```

CASE STUDY 7: PASSWORD VALIDATION ENGINE

String Method Traversal

Problem Statement: Evaluate user password strings against standard policy rules: minimum length, uppercase, lowercase, numbers, and symbols.

LOGIC & STRING PARSING FLOWCHART



PYTHON SOURCE CODE

```
password = input("Enter password: ") checks = { "Min 8 Characters": len(password) >= 8, "Uppercase Letter": any(ch.isupper() for ch in password), "Lowercase Letter": any(ch.islower() for ch in password), "Digit Character": any(ch.isdigit() for ch in password), "Special Symbol": any(not ch.isalnum() for ch in password) } for rule, passed in checks.items(): print(f" [{'PASSED' if passed else 'FAILED'}] {rule}") if all(checks.values()): print("Result: VALID PASSWORD") else: print("Result: INVALID PASSWORD")
```

EXECUTION OUTPUT

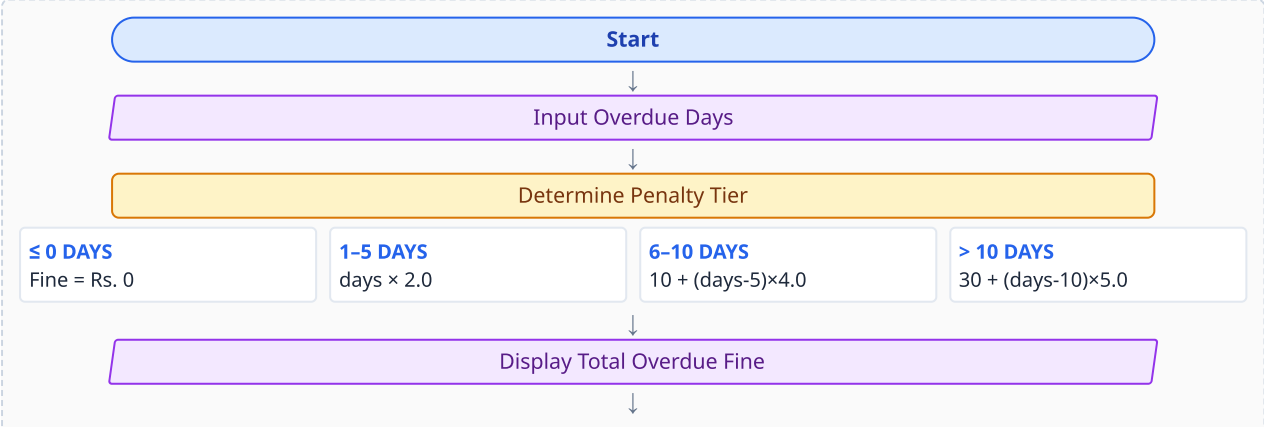
```
Enter password: SecurePass123! [PASSED] Min 8 Chars | [PASSED] Uppercase | [PASSED] Lowercase | [PASSED] Digit | [PASSED] Special Result: VALID PASSWORD
```

CASE STUDY 8: LIBRARY LATE FINE CALCULATOR

Tiered Penalties

Problem Statement: Calculate late book return fines using progressive tier structures (Rs.2/day for 1-5 days, Rs.4/day for 6-10 days, Rs.5/day beyond 10 days).

LOGIC & SLAB FLOWCHART





PYTHON SOURCE CODE

```
late_days = int(input("Enter overdue days: ")) if late_days <= 0: fine = 0.0 elif late_days <= 5: fine = late_days * 2.0 elif late_days <= 10: fine = (5 * 2.0) + ((late_days - 5) * 4.0) else: fine = (5 * 2.0) + (5 * 4.0) + ((late_days - 10) * 5.0) print(f"Overdue: {late_days} Days | Fine Due: Rs. {fine:.2f}")
```

EXECUTION OUTPUT

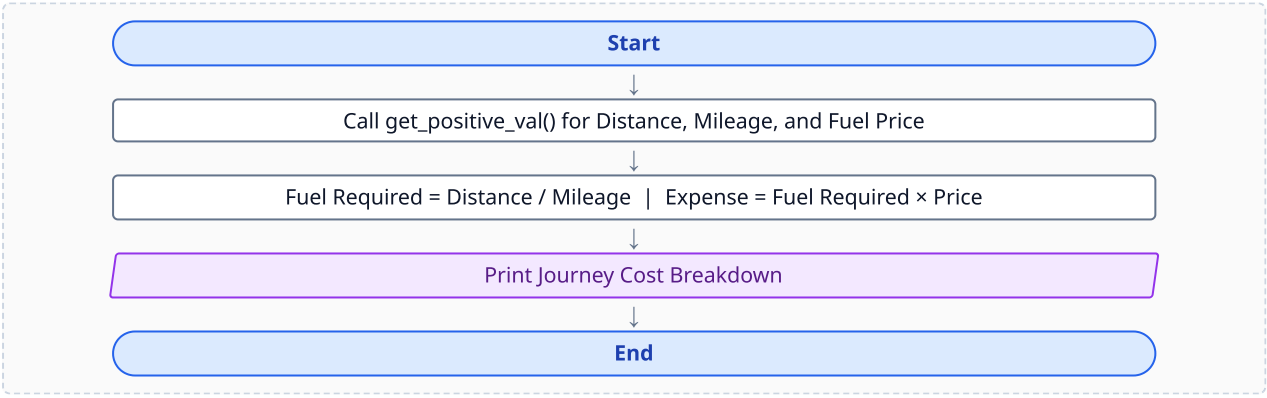
```
Enter number of overdue days: 12 Overdue Duration: 12 Days | Total Fine Due: Rs. 40.00
```

CASE STUDY 9: ROUTE COST & FUEL CALCULATOR

Function Operations

Problem Statement: Compute total fuel required and journey monetary costs using modular positive input validation functions.

LOGIC & FUNCTION FLOWCHART



PYTHON SOURCE CODE

```
def get_positive_val(prompt): while True: try: val = float(input(prompt)) if val > 0: return val except ValueError: pass distance = get_positive_val("Enter distance (km): ") mileage = get_positive_val("Enter mileage (km/l): ") price = get_positive_val("Enter fuel price (Rs./l): ") fuel_needed = distance / mileage total_cost = fuel_needed * price print(f"Distance: {distance:.2f}km | Fuel: {fuel_needed:.2f}L | Cost: Rs.{total_cost:.2f}")
```

EXECUTION OUTPUT

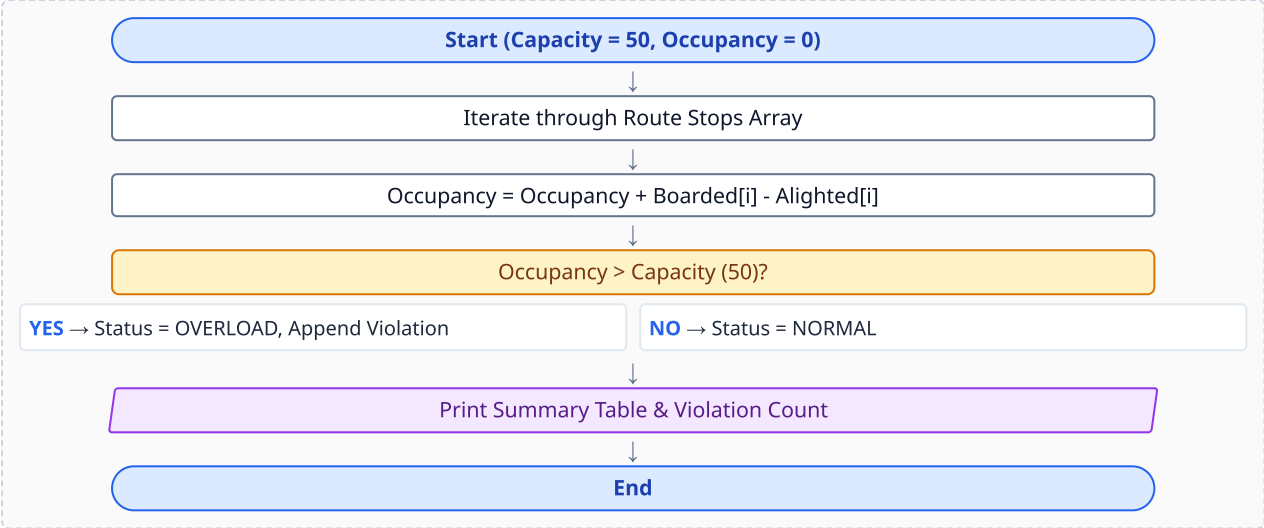
```
Distance: 450 km | Mileage: 15 km/l | Fuel Price: Rs. 102 Fuel Required: 30.00 Liters | Total Expense: Rs. 3060.00
```

CASE STUDY 10: BUS OCCUPANCY SIMULATION

List Iteration & Flags

Problem Statement: Track stop-by-stop passenger boarding/alighting along a route and flag overload violations against a max capacity threshold (50).

LOGIC & ITERATION FLOWCHART



PYTHON SOURCE CODE

```
CAPACITY = 50 stops = ["Terminal A", "Stop 1", "Stop 2", "Stop 3", "Terminal B"] boarded = [35, 20, 15, 5, 0] alighted = [0, 5, 8, 22, 40] occupancy = 0 violations = [] for i in range(len(stops)): occupancy += boarded[i] - alighted[i] status = "NORMAL" if occupancy > CAPACITY: status = f"OVERLOAD (+{occupancy - CAPACITY})" violations.append((stops[i], occupancy)) print(f"{stops[i]:<12} | Occ: {occupancy:<2} | Status: {status}") print("Total Overload Violations:", len(violations))
```

EXECUTION OUTPUT

```
Terminal A: 35 (NORMAL) | Stop 1: 50 (NORMAL) | Stop 2: 57 (OVERLOAD +7) Stop 3: 40 (NORMAL) | Terminal B: 0 (NORMAL) | Violations Recorded: 1
```